

Cahier des charges

GuitarFlow - Projet de transcription audio vers MIDI

1. Table des matières

- 1. Table des matières
- 2. Présentation du projet
 - 2.1. Contexte
 - 2.2. Objectifs du projet
 - 2.3. Objectifs de performance
- 3. Périmètre du projet
 - 3.1. Fonctionnalités incluses
 - 3.2. Fonctionnalités hors périmètre
- 4. Utilisateurs cibles
 - 4.1. Profils utilisateurs
 - 4.2. Cas d'usage principal
- 5. Contraintes budgétaires
- 6. Exigences fonctionnelles
 - 6.1. Gestion des fichiers audio
 - 6.2. Préprocessing audio
 - 6.3. Extraction des features
 - 6.4. Modélisation
 - 6.5. Génération MIDI
 - 6.6. Interface utilisateur
 - 6.7. API d'inférence
- 7. Exigences techniques
 - 7.1. Stack technique cible
 - 7.2. Contraintes techniques
 - 7.3. Qualité logicielle
- 8. Données et gouvernance
 - 8.1. Sources de données
 - 8.2. Gouvernance des données
 - 8.3. Contraintes réglementaires
- 9. Exigences de performance
 - 9.1. Performance ML
 - 9.2. Performance système
- 10. Livrables attendus
 - 10.1. Livrables techniques
 - 10.2. Livrables documentaires
- 11. Planning prévisionnel
- 12. Critères de recette
- 13. Risques identifiés
- 14. Perspectives d'évolution

- [15. Validation](#)

2. Présentation du projet

2.1. Contexte

GuitarFlow souhaite développer une solution permettant de convertir automatiquement un enregistrement audio de guitare en fichier MIDI exploitable dans des logiciels de MAO (Musique Assistée par Ordinateur).

Le projet vise à automatiser les tâches de retranscription musicale afin de réduire le temps de traitement manuel et de faciliter la création de contenus pédagogiques et musicaux.

2.2. Objectifs du projet

Le système devra permettre :

- l'import d'un fichier audio guitare au format .wav,
- la génération d'un fichier MIDI,
- la visualisation des notes détectées sous forme de piano-roll,
- éventuellement la génération d'une partition ou d'une tablature,
- l'exposition du service via une API et une interface web.

2.3. Objectifs de performance

Indicateur	Objectif cible
F1-score transcription polyphonique	≥ 90 %
Temps d'inférence	< 10 s pour 1 min audio
Disponibilité du prototype	Démonstration fonctionnelle avant le 31/07/2026
Déploiement	API accessible via interface web
Taux de correction manuelle	< 10 %

3. Périmètre du projet

3.1. Fonctionnalités incluses

Le système devra :

- accepter des fichiers audio .wav,
- supporter les guitares acoustiques et électriques,
- fonctionner avec accordage standard EADGBE,
- produire un fichier MIDI téléchargeable,
- afficher une visualisation piano-roll,
- générer si possible une partition ou tablature,
- proposer une API REST d'inférence,

- être déployé sur Hugging Face Spaces,
- être conteneurisé via Docker.

3.2. Fonctionnalités hors périmètre

Le système ne devra pas gérer :

- les autres instruments,
- les accordages alternatifs,
- la transcription temps réel,
- les techniques avancées de jeu (bend, vibrato, slide, hammer-on / pull-off, ...),
- les applications mobiles,
- les architectures cloud.

4. Utilisateurs cibles

4.1. Profils utilisateurs

Le système cible :

- professeurs de guitare,
- créateurs de contenu pédagogique,
- musiciens amateurs,
- compositeurs.

4.2. Cas d'usage principal

1. L'utilisateur charge un fichier .wav.
2. Le système traite le signal audio.
3. Le modèle détecte les notes.
4. Le système génère :
 - un fichier MIDI,
 - un piano-roll,
 - éventuellement une partition ou tablature.
5. L'utilisateur télécharge le résultat.

5. Contraintes budgétaires

Le projet s'inscrit dans une logique de prototypage à faible coût.

La phase 1 devra privilégier :

- les solutions open source,
- les infrastructures gratuites ou locales,
- les composants faiblement coûteux en calcul.

Aucune infrastructure cloud industrielle n'est prévue dans le cadre de la phase 1.

6. Exigences fonctionnelles

6.1. Gestion des fichiers audio

Le système devra :

- accepter les fichiers .wav,
- vérifier le format d'entrée,
- rejeter les fichiers invalides,
- gérer des fichiers jusqu'à 3 minutes.

6.2. Préprocessing audio

La pipeline devra inclure :

- conversion mono,
- rééchantillonnage,
- normalisation des signaux,
- réduction du bruit,
- suppression des silences.

Le système devra garantir l'homogénéité des signaux avant extraction des features.

6.3. Extraction des features

Le système devra permettre l'extraction des représentations suivantes :

- STFT,
- Mel Spectrogram,
- MFCC,
- CQT,
- chromagrammes.

Les features devront être configurables afin de permettre des expérimentations comparatives.

6.4. Modélisation

Le système devra permettre l'évaluation de plusieurs approches :

- baseline simple,
- MLP,
- architectures convolutionnelles récurrentes (RCNN).

Les modèles devront être entraînaables et versionnés.

6.5. Génération MIDI

Le système devra :

- convertir les prédictions du modèle en événements MIDI,

- générer un fichier .mid exploitable dans un logiciel de MAO.

6.6. Interface utilisateur

L'interface web devra permettre :

- le téléchargement du fichier audio,
- le lancement du traitement,
- le téléchargement du fichier MIDI,
- l'affichage du piano-roll
- éventuellement le téléchargement d'une partition ou tablature.

6.7. API d'inférence

Le système devra exposer une API REST permettant :

- l'envoi d'un fichier audio,
- la récupération des résultats,
- l'obtention des métadonnées de traitement.

7. Exigences techniques

7.1. Stack technique cible

Domaine	Technologie envisagée
Langage principal	Python
Deep Learning	tensorflow / keras
API	FastAPI
Conteneurisation	Docker
Déploiement	Hugging Face Spaces
Experiment tracking	MLflow
Gestion des dépendances	uv
CI/CD	GitHub Actions

7.2. Contraintes techniques

Le système devra :

- permettre une exécution locale,
- être reproductible,
- permettre la réexécution des expérimentations.

7.3. Qualité logicielle

Le projet devra inclure :

- structuration modulaire du code,
- tests,
- logging,
- gestion des erreurs,
- validation qualité (lint + format + typing),
- documentation technique minimale,
- Dockerfile fonctionnel.

8. Données et gouvernance

8.1. Sources de données

Les datasets suivants pourront être utilisés :

- [GuitarSet](#),
- [IDMT-SMT-Guitar](#).

8.2. Gouvernance des données

Le système devra assurer :

- séparation train / validation / test,
- versioning datasets,
- traçabilité des expérimentations,
- conservation des métriques,
- reproductibilité des entraînements.

8.3. Contraintes réglementaires

Une vérification préalable des licences datasets devra être réalisée avant exploitation.

9. Exigences de performance

9.1. Performance ML

KPI	Objectif
F1-score	≥ 90 %
Precision	≥ 90 %
Recall	≥ 90 %

9.2. Performance système

KPI	Objectif
Temps moyen d'inférence	< 10 s
Disponibilité démonstrateur	100 % pendant soutenance

10. Livrables attendus

10.1. Livrables techniques

Le projet devra fournir :

- pipeline d'ingestion,
- pipeline de preprocessing,
- pipeline d'entraînement,
- modèle entraîné et évalué,
- API REST,
- conteneur Docker,
- interface Hugging Face.

10.2. Livrables documentaires

Le projet devra fournir :

- note de cadrage,
- cahier des charges,
- documentation technique,
- documentation d'installation,
- support de présentation.

11. Planning prévisionnel

Etape	Activité
1	Ingestion des données
2	Analyse des données
3	Préprocessing audio
4	Feature engineering
5	Benchmark modèles
6	Implémentation pipeline ML
7	Développement API
8	Déploiement Hugging Face
9	Validation et démonstration

12. Critères de recette

Le projet sera considéré conforme si :

- l'API répond correctement,
- un fichier MIDI valide est généré,
- les performances minimales sont atteintes,
- le prototype est démontrable,
- les expérimentations sont reproductibles.

13. Risques identifiés

Risque	Impact	Mitigation
Restrictions de licence sur les datasets	Élevé	Validation juridique préalable
Données insuffisantes	Élevé	Data augmentation
Polyphonie complexe	Élevé	Benchmark multi-modèles
Temps d'inférence	Moyen	Optimisation pipeline
Variabilité audio	Moyen	Préprocessing robuste

14. Perspectives d'évolution

Les évolutions futures envisagées sont :

- support multi-instruments,
- transcription temps réel,
- détection avancée des techniques de jeu,
- amélioration polyphonique,
- architecture cloud scalable,
- application mobile.

15. Validation

Le présent cahier des charges servira de référence pour :

- le développement du prototype,
- le suivi du projet,
- la validation des livrables,
- l'évaluation de la conformité fonctionnelle et technique du démonstrateur.

Auteur : Damien DESSAUX